Kevin Küster

Zusammenfassung: Neumann-Architektur

Neumann-Architektur

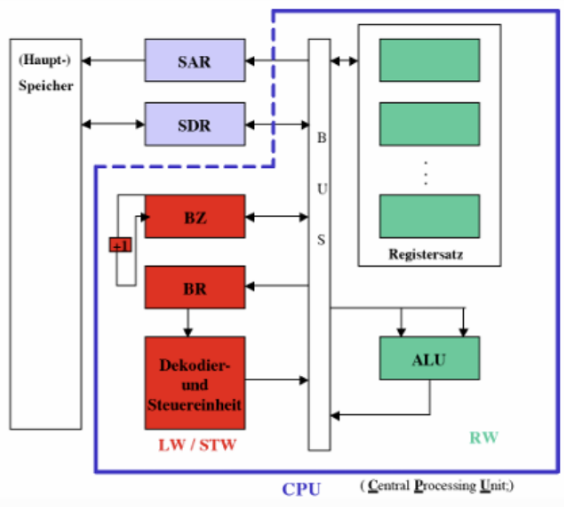
Von Neumann-Prinzipien

1. Rechner besteht aus 4 Hauptkomponenten:
   (i) Rechenwerk
   => führt arithmetische & logische Operationen aus
   (ii) Steuerwerk (auch Leitwerk)
   => Interpretation des Programms & Steuerung des Ablaufs anderer Werke
   (iii) Speicherwerk
   => Hauptspeicher und Arbeitsspeicher für Programme & Daten
   (iv) Ein-/Ausgabewerk
   => Kommunikation mit Umwelt & Sekundärspeicher -> Langzeitspeicher
2. Programm und Daten stehen in demselben speicher => Hauptspeicher
   ↳ können von der Maschine (während der Ausführung) verändert werden
3. Struktur des Rechners ist unabhängig von dem bearbeitenden Problem (Programmsteuerung)
4. Hauptspeicher ist in Zellen gleicher Größe geteilt, die durch fortlaufende Nummern (Adressen) gekennzeichnet sind

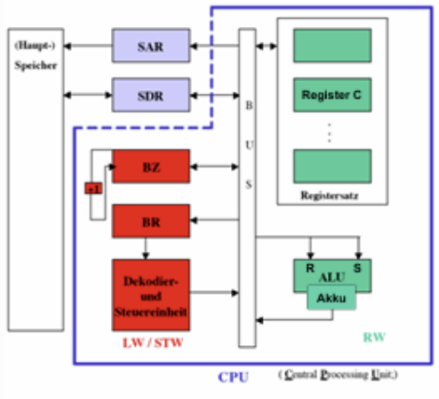


5. Das Programm besteht aus einer Folge von Befehlen (Maschinenbefehle), die i.A. nacheinander ausgeführt werden (=> Prinzip der Sequenzialität)
6. Von der strikt sequenziellen Ausführung von Maschinenbefehlen kann durch bedingte oder unbedingte Sprünge abgewichen werden.
   => Sprung im Programm (andere Zelle im Hauptspeicher)
   ↳ abhängig von gespeicherten Werten; Summe dieser Werte: Status der Maschine
7. Maschinen verwenden das Dualzahlensystem

CPU-Aufbau & Funktion



SAR: Speicher Adressregister
SDR: Speicher Datenregister
LW / STW: Leitwerk / Steuerwerk
BZ: Befehlszähler
BR: Befehlsregister
+1: Steuerung der Befehlsausführung
RW: Rechenwerk
ALU: Arithmetisch / Logische Einheit
BUS: Datensammelleitung zur Verbindung der Teilwerke der CPU



Beispiel: Register C um 1 erhöhen
Reg.C = Reg.C + 1;

Schrittweise Befehlsabarbeitung:

1. Befehlsholphase (BH):
   SAR ⇐ BZ
   SDR ⇐ Hauptspeicher(SAR)
   BR ⇐ SDR
2. Dekodierphase (DE)
   Dekodierung(BR) /* Inkrement(ALU.R) */
   /* Source, Target: Reg.C */
3. Operandenholphase (OH)
   - - - /* Datum liegt in Reg.C bereits vor */
4. Ausführungsphase (AP)
   ALU.R ⇐ Reg.C
   ALU.Akku ⇐ Inkrement(ALU.R)
5. Rückschreibephase (RS)
   Reg.C ⇐ ALU.Akku
6. Adressierungsphase (AD)
   BZ ⇐ BZ+1;

Befehlsausführungszyklus

Beschreibt die gemeinsame Arbeitsweise von Leitwerk und Rechenwerk => Befehlszyklus:

1. Befehlsholphase (BH)
   Auf Basis des Inhalts des Befehlzählers (BZ) wird über den Systembus (SAR, SDR) der nächste auszuführende Maschinenbefehl (OpCode) aus dem Hauptspeicher in das Befehlsregister (BR) geladen.
2. Dekodierphase (DK)
   Nachdem der Befehl im Befehlsregister steht, beginnt die Interpretation durch die Dekodierung des OpCodes
   ↳ generiert weitere Steuersignale und Ausführung einer elementaren Steuersequenz
   (=> Steuerwerk, Mikroprogramm)
3. Operandenholphase (OH)
   Operanden, die im Adressteil der Operation spezifiziert, werden (sofern nötig) aus dem Hauptspeicher geholt und in einem Register des Rechenwerks bereit gestellt
4. Ausführungsphase (AP)
   Operanden werden aus einem Register geholt und an die ALU durchgeschaltet, die Funktion auf die Operanden ausgeführt und das Ergebnis in den Akkumulator übertragen
5. Rückschreibephase (RS)
   Das in der Ausführungsphase erzeugte Ergebnis wird in die hierfür bestimmten Speicherzellen / Register geschrieben
6. Adressierungsphase (AD)
   Der Befehlzähler wird erhöht

Engpässe

(i) Befehle & Daten stehen gleichzeitig, ununterscheidbar nebeneinander im Speicher und werden über die gleichen Wege gelesen bzw. geschrieben (Speicherschnittstelle, SAR, SDR). Zugriffe behindern sich gegenseitig → Von-Neumann-Bottleneck; one-word-at-a-time-thinking | Sequentialismus

(ii) Verwendung von Speicherworten gleicher Länge für Daten & Befehle ⇒ Mehrfachzugriffe

(iii) Keine Möglichkeit der gleichzeitigen Ausführung von mehreren Programmschritten. Abarbeitung einzelner Programmschritte in einem starren, sequenziellen Befehlsausführungszyklus

(iv) Feste Befehlsabfolge & Steuerung in nur einer Ebene, d.h. vorhanden sein von nur einem Befehlszähler und Befehlsregister.
⇒ Umschalten (Kontextwechsel) bei Unterprogrammaufrufen, Multitasking, Multiusorbetrieb aufwendig & ineffizient

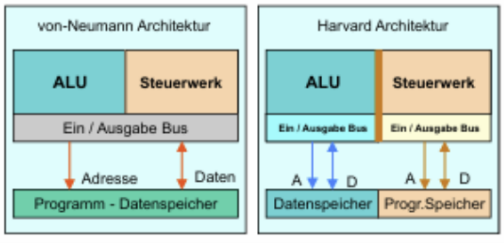
---

Von-Neumann-Bottleneck

Engpässe (i)

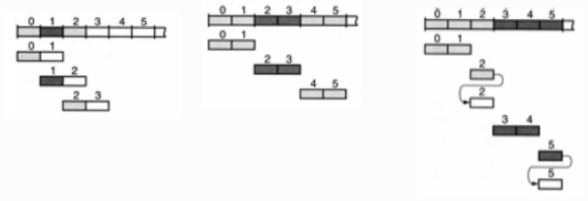
Abweichungen

Auf den unteren Ebenen der Speicherhierarchie werden getrennte Speicher für Daten und Befehle verwendet; Gleichzeitiger Zugriff über getrennte Busse möglich
(→ Harvard Architektur, → RISC, getrennte Caches)



Vereinheitlichung der Wortbreite für Daten & Befehle zur Vermeidung / Verringerung von Mehrfachzugriffen:

Programm: 1-Bit-Befehle 2-Bit-Befehle 3-Bit-Befehle



Optimierung

Ursprüngliche, zweistufige Speicherhierarchie (Haupt- & Hintergrundspeicher) ⇒ Mehrstufige Speicherhierarchie (Spatial & Temporal Locality)
Vervielfachung einzelner / mehrerer Teile oder Werke → Steigerung Effizienz / Rechenleistung & Zuverlässigkeit:
↳ zusätzliche Busse & E/A-Werke ⇒ Kommunikations- & Peripherieleistung verbessern
↳ Verwendung mehrerer, parallel arbeitender Rechenwerke (→ Parallelverarbeitung, Multithreading, Multiprocessing) ⇒ Erhöhung der Rechenleistung

Einführung eines Universalregistersatzes (Register Renaming, Register Pointer, Register Windows) & Bereitstellung von mehreren Befehlszählern & Befehlsregistern. ⇒ Kontextwechsel effizienter & Steuerungsabläufe (für Befehlsausführung) vereinfacht

---